

Các chủ đề nâng cao

trong thiết kế hệ thống DB

AGENDA

01

Tìm kiếm trên dữ liệu lớn

02

Replication & Sharding

03

ACID vs BASE

04

Giao dịch phân tán

05

Bài tập áp dụng

Tìm kiếm trên dữ liệu lớn

Thách thức khi tìm kiếm dữ liệu lớn

Tìm kiếm trên hàng tỉ bản ghi bằng `LIKE '%keyword%'` là không khả thi trong môi trường production.

- ☕ **Full table scan** với `LIKE` có độ phức tạp $O(n \times m)$ — cực kỳ chậm
- ☕ Dữ liệu văn bản không có cấu trúc cố định (bài viết, mô tả sản phẩm)
- ☕ Người dùng kỳ vọng kết quả **dưới 200ms** dù dataset có hàng trăm triệu bản ghi
- ☕ Cần hỗ trợ: tìm gần đúng, gợi ý, tìm đa ngôn ngữ

Inverted Index – cơ chế hoạt động

Inverted Index ánh xạ từng từ (**term**) tới danh sách các document chứa từ đó — đảo ngược mối quan hệ thông thường.

- ☕ Thay vì "document → danh sách từ", lưu "từ → danh sách document ID"
- ☕ **Tokenization**: tách văn bản thành các token (từ đơn)
- ☕ **Normalization**: lowercase, bỏ dấu, loại stop words ("là", "và", "the")

Tìm kiếm = tra cứu hash map → $O(1)$ thay vì $O(n)$

Ví dụ

Documents:

- Doc 1: "database lớn cần index"
- Doc 2: "index giúp tìm kiếm nhanh"
- Doc 3: "tìm kiếm database hiệu quả"

Inverted Index:

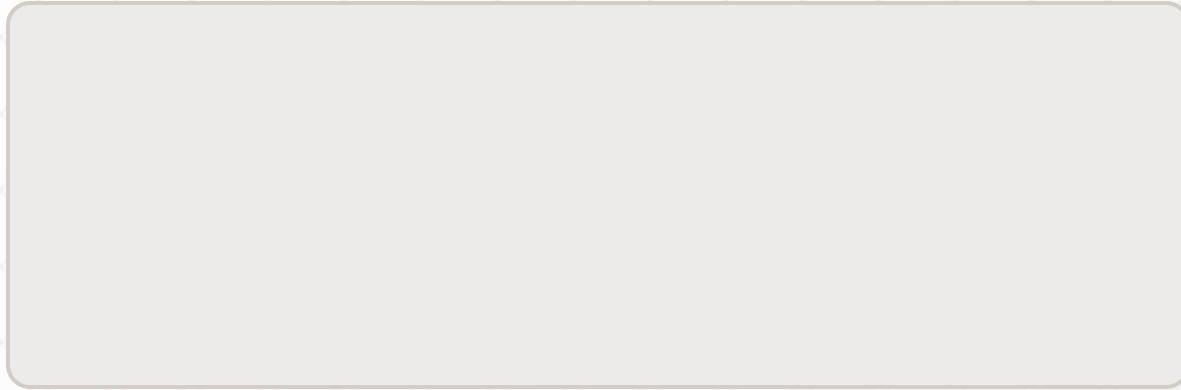
Term	Doc IDs
database	1, 3
index	1, 2
tìm kiếm	2, 3
nhanh	2

Full-text Search với Elasticsearch

Elasticsearch là distributed search engine xây dựng trên Apache Lucene, cung cấp full-text search với độ trễ thấp ở quy mô lớn.

- ☕ Lưu trữ document dạng **JSON**, tự động đánh index khi ingest
- ☕ **Shard**: chia index thành nhiều phần, phân tán trên nhiều node
- ☕ **Replica**: sao chép shard để tăng availability và read throughput
- ☕ **Relevance scoring**: dùng TF-IDF và BM25 để xếp hạng kết quả
- ☕ Hỗ trợ **analyzer** tùy chỉnh: tokenizer, filter, stemmer

Ví dụ: Truy vấn Elasticsearch



-  `^3` — tăng trọng số field `name` gấp 3 lần
-  `fuzziness: AUTO` — tự động khớp từ gần đúng
-  `highlight` — đánh dấu từ khóa trong kết quả trả về

Fuzzy Search — tìm kiếm gần đúng

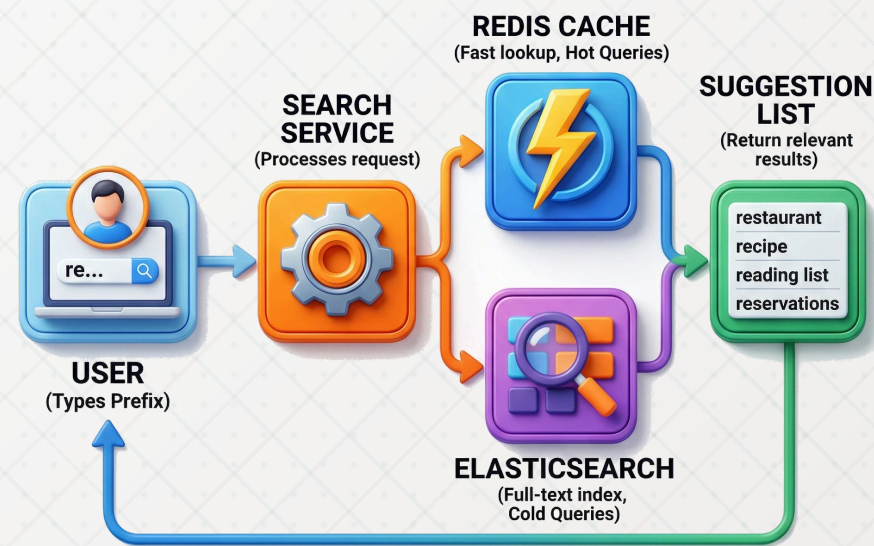
Fuzzy Search tìm kiếm các chuỗi gần giống với từ khóa, dựa trên khoảng cách chỉnh sửa (**Levenshtein Distance**).

- ☕ **Levenshtein Distance**: số thao tác tối thiểu (thêm, xóa, thay) để biến chuỗi A thành B
- ☕ `"pytohn"` → `"python"`: distance = 2 (hoán vị 2 ký tự)
- ☕ Hữu ích khi người dùng gõ sai chính tả hoặc thiếu dấu
- ☕ Elasticsearch hỗ trợ `fuzziness: 1` hoặc `AUTO`

Typeahead / Autocomplete

Typeahead gợi ý từ khóa hoàn chỉnh khi người dùng đang gõ, dùng **prefix matching** hoặc **n-gram index**.

- ☕ **Prefix matching:** "iph" → "iphone 15", "iphone 14"
- ☕ **N-gram:** chia token thành chuỗi con — hỗ trợ tìm giữa chuỗi
- ☕ Kết quả phải trả về **< 100ms** để không ảnh hưởng UX
- ☕ Cache kết quả phổ biến tại Redis để giảm tải Elasticsearch



Replication & Sharding

Tại sao cần Replication?

Replication là quá trình sao chép dữ liệu từ một node sang các node khác để tăng **availability** và **read throughput**.

- ☕ **Single point of failure**: nếu chỉ có 1 DB node → hệ thống sập khi node đó lỗi
- ☕ **Read scalability**: nhiều replica phục vụ read → giảm tải master
- ☕ **Geographic distribution**: đặt replica gần người dùng → giảm latency
- ☕ **Disaster recovery**: replica ở data center khác → khôi phục sau thảm họa

Master-Slave Replication

Một **master** nhận tất cả write, sao chép sang một hoặc nhiều **slave** chỉ phục vụ read.

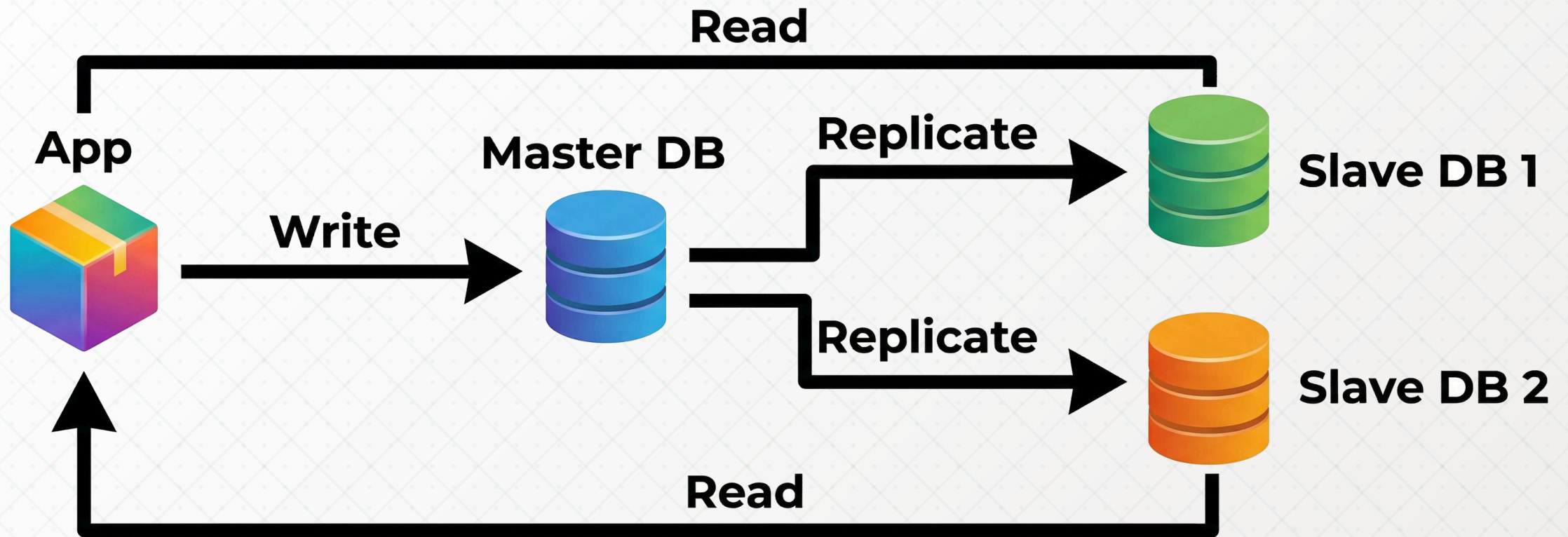
Ưu điểm:

- Đơn giản, dễ triển khai
- Slave scale read tuyến tính
- Master tập trung xử lý write → nhất quán

Nhược điểm:

- Master là single point of failure cho write
- **Replication lag**: slave có thể đọc dữ liệu cũ
- Failover thủ công tốn thời gian

Master-Slave Replication System Diagram



Master-Master Replication

Hai (hoặc nhiều) node **đều** nhận write và sao chép lẫn nhau — không có node nào là slave.

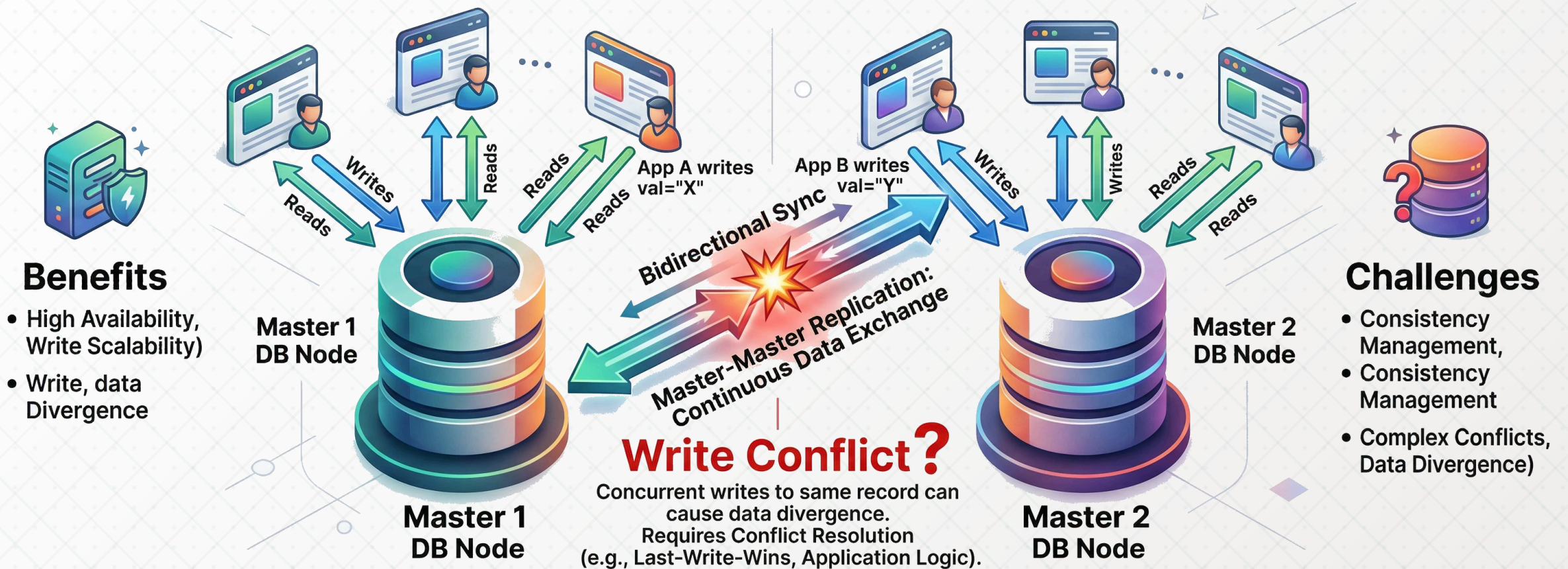
Ưu điểm:

- ☕ Không có single point of failure cho write
- ☕ Write có thể phân tán địa lý
- ☕ Tự động failover dễ hơn

Nhược điểm:

- ☕ **Write conflict:** hai node ghi cùng row đồng thời → xung đột
- ☕ Phức tạp hơn, cần conflict resolution strategy
- ☕ Nguy cơ mất tính nhất quán (**consistency**)

MASTER-MASTER REPLICATION ARCHITECTURE & KEY CHALLENGES



So sánh Master-Slave vs Master-Master

Master-Slave

- ☕ Write: chỉ master
- ☕ Read: master + slave
- ☕ Consistency: cao
- ☕ Complexity: thấp
- ☕ Failover: thủ công / bán tự động
- ☕ Dùng khi: read-heavy, cần consistency

Master-Master

- ☕ Write: tất cả node
- ☕ Read: tất cả node
- ☕ Consistency: thấp hơn
- ☕ Complexity: cao
- ☕ Failover: tự động
- ☕ Dùng khi: write cần HA, multi-region

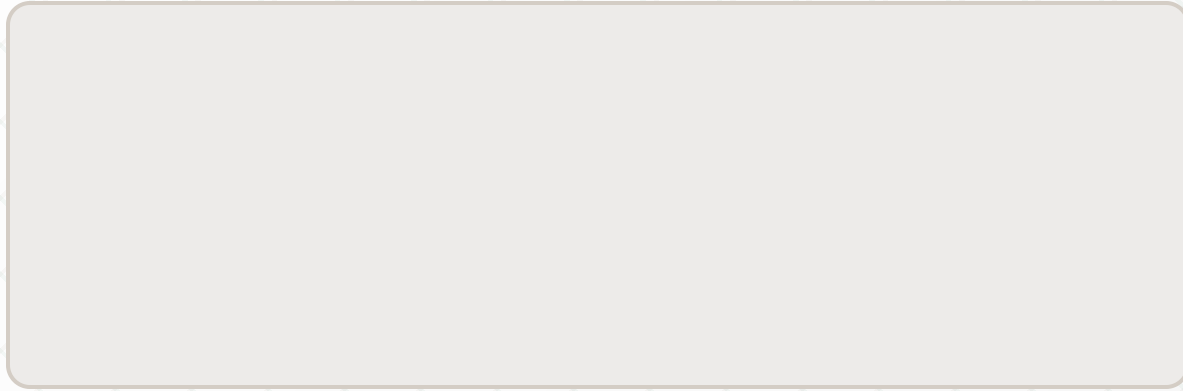
ACID vs BASE

ACID Transactions – 4 tính chất

ACID đảm bảo tính toàn vẹn dữ liệu tuyệt đối — mọi transaction hoặc thành công hoàn toàn, hoặc không xảy ra gì.

- ☕ **Atomicity** (Tính nguyên tử): transaction là một đơn vị — tất cả thành công hoặc tất cả rollback
- ☕ **Consistency** (Nhất quán): DB luôn ở trạng thái hợp lệ trước và sau transaction
- ☕ **Isolation** (Cô lập): các transaction song song không can thiệp lẫn nhau
- ☕ **Durability** (Bền vững): dữ liệu đã commit không bao giờ bị mất, dù server crash

Ví dụ: ACID trong chuyển tiền ngân hàng



- Nếu server crash sau `UPDATE A` nhưng trước `UPDATE B` → **tự động ROLLBACK**
- Không thể xảy ra trạng thái: A bị trừ tiền nhưng B không nhận được

BASE Transactions – 3 tính chất

BASE ưu tiên **availability** và **performance** hơn consistency tuyệt đối — chấp nhận dữ liệu *eventually* nhất quán.

- ☕ **Basically Available**: hệ thống luôn phản hồi, dù có thể trả về dữ liệu cũ
- ☕ **Soft state**: trạng thái hệ thống có thể thay đổi theo thời gian dù không có input mới (do propagation)
- ☕ **Eventually Consistent**: dữ liệu sẽ nhất quán *cuối cùng* — không phải ngay lập tức
- ☕ Phổ biến trong **NoSQL** (Cassandra, DynamoDB, CouchDB)

So sánh ACID vs BASE

Lựa chọn giữa ACID và BASE là đánh đổi giữa **consistency** và **availability/performance**.

ACID

- Consistency: **tuyệt đối**
- Availability: có thể giảm (locks)
- Latency: cao hơn
- Scale: vertical chủ yếu
- Dùng: tài chính, y tế, ngân hàng
- VD: PostgreSQL, MySQL, Oracle

BASE

- Consistency: **eventually**
- Availability: **cao**
- Latency: thấp hơn
- Scale: horizontal tốt
- Dùng: social media, analytics, IoT
- VD: Cassandra, DynamoDB, MongoDB

Khi nào dùng ACID? Khi nào dùng BASE?

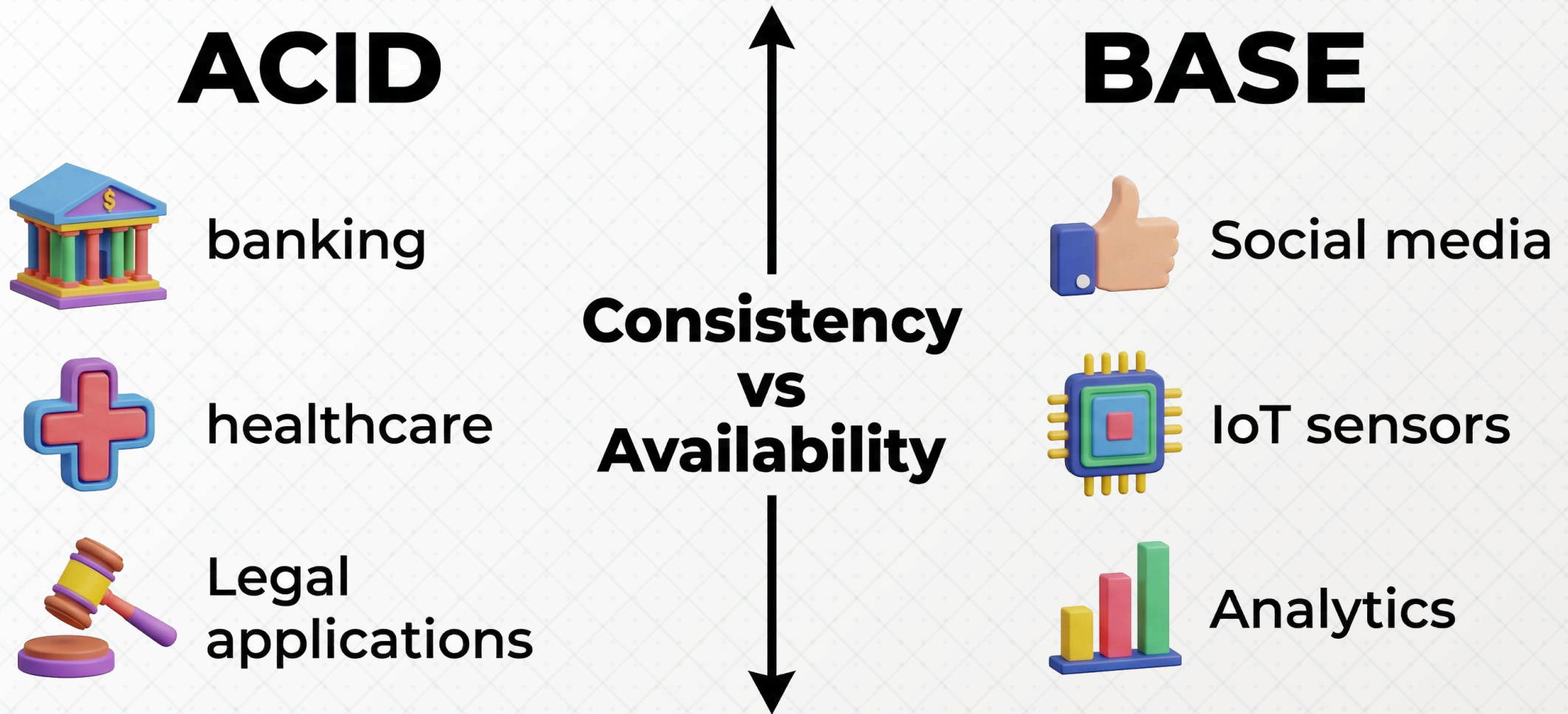
Chọn mô hình phù hợp với **yêu cầu nghiệp vụ**, không phải theo xu hướng công nghệ.

Dùng ACID khi:

- ☉ Dữ liệu tài chính, y tế, pháp lý — sai sót gây hậu quả nghiêm trọng
- ☉ Cần foreign key constraint và relational integrity
- ☉ Write volume không quá lớn (< vài nghìn TPS)

Dùng BASE khi:

- ☉ Scale hàng triệu write/giây (logs, events, IoT sensor data)
- ☉ Chấp nhận stale read (feed mạng xã hội, số like, view count)
- ☉ Cần geo-distributed write với latency thấp



Giao dịch phân tán

Thách thức của giao dịch phân tán

Trong hệ thống phân tán, một *business transaction* có thể span nhiều service và database — đảm bảo ACID trở nên *cực kỳ khó*.

- ☕ **Network partition:** các node không thể liên lạc với nhau (CAP theorem)
- ☕ **Partial failure:** một số service thành công, một số fail → dữ liệu không nhất quán
- ☕ **No global clock:** các node không có đồng hồ chung → khó xác định thứ tự sự kiện
- ☕ **2PC blocking:** giao thức cổ điển có thể block toàn hệ thống nếu coordinator crash

Two-Phase Commit (2PC)

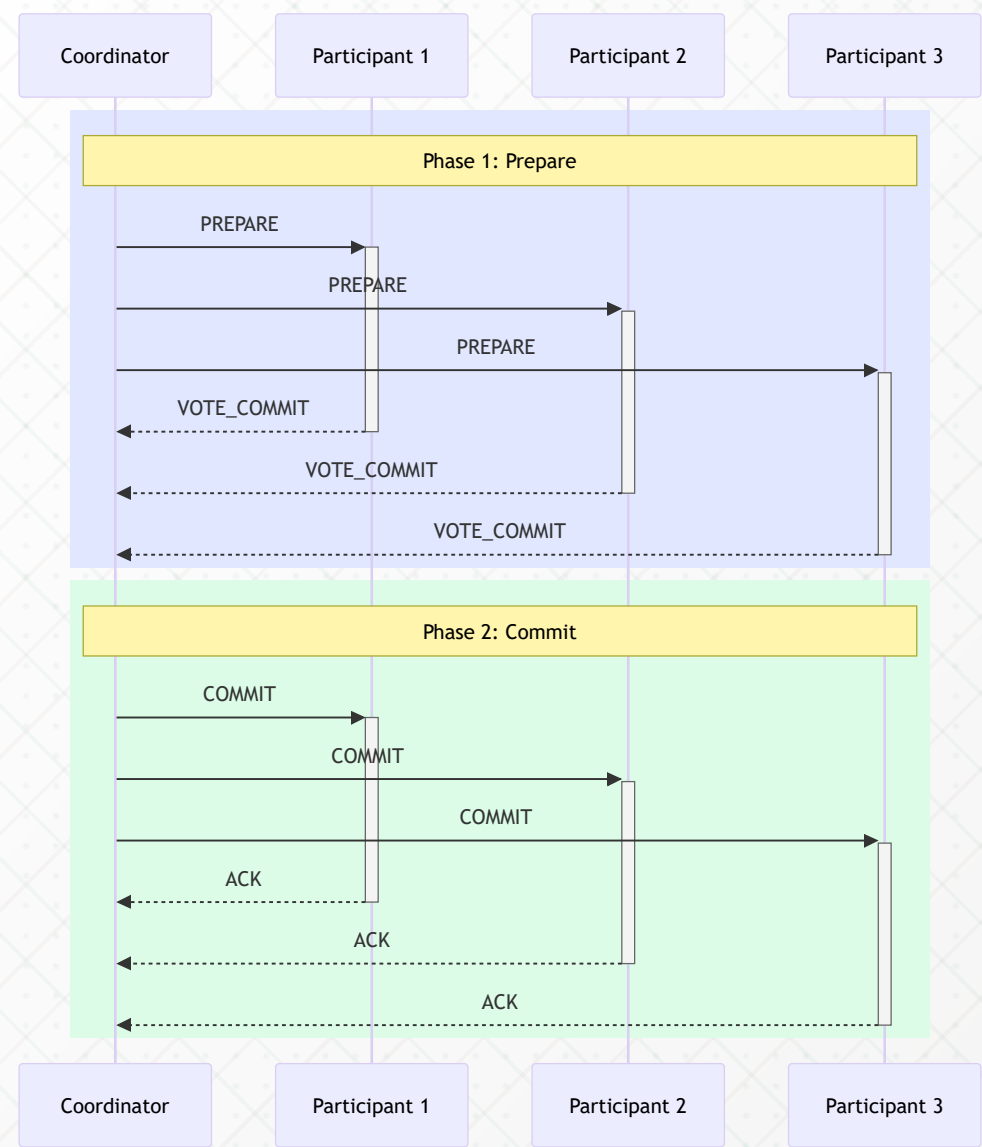
2PC dùng một **Coordinator** trung tâm điều phối 2 pha — *Prepare* và *Commit* — để đảm bảo tất cả participant đồng ý trước khi commit.

Pha 1 — Prepare:

- Coordinator gửi **PREPARE** tới tất cả participant
- Mỗi participant chuẩn bị transaction, ghi redo log, trả lời **VOTE_COMMIT** hoặc **VOTE_ABORT**

Pha 2 — Commit:

- Nếu **tất cả** vote **VOTE_COMMIT** → Coordinator gửi **COMMIT**
- Nếu **bất kỳ** vote **VOTE_ABORT** → Coordinator gửi **ABORT** tới tất cả



Hạn chế của 2PC

2PC có vấn đề **blocking** nghiêm trọng: nếu Coordinator crash sau pha Prepare, toàn hệ thống bị treo.

- ☕ **Blocking protocol:** participant giữ lock trong suốt quá trình → block resource
- ☕ **Single point of failure:** Coordinator crash → tất cả participant chờ vô thời hạn
- ☕ **Performance:** 2 round-trip network cho mỗi transaction → latency cao
- ☕ **Không phù hợp** cho microservices với hundreds of service instances

Three-Phase Commit (3PC)

3PC thêm pha **Pre-Commit** giữa Prepare và Commit, cho phép participant tự quyết định nếu Coordinator fail — giải quyết blocking problem.

Ba pha:

1. **CanCommit**: Coordinator hỏi "bạn có thể commit không?" — không lock resource
2. **PreCommit**: nếu tất cả đồng ý → gửi `PREPARE_COMMIT`, participant giữ lock
3. **DoCommit**: Coordinator gửi `COMMIT` → participant thực hiện commit

Cải tiến so với 2PC:

- Participant biết rằng *tất cả đã sẵn sàng* trước khi lock resource
- Nếu Coordinator timeout ở pha 3, participant có thể tự commit (safe)
- Giảm blocking khi Coordinator crash

So sánh 2PC vs 3PC

Cả hai đều đảm bảo atomicity nhưng có đánh đổi khác nhau về blocking và complexity.

2PC

- ☕ Số pha: 2
- ☕ Blocking: **có** (coordinator fail)
- ☕ Message count: $2N + 2$
- ☕ Complexity: trung bình
- ☕ Recovery: cần manual hoặc log
- ☕ Dùng phổ biến trong: XA, JDBC

3PC

- ☕ Số pha: 3
- ☕ Blocking: **ít hơn**
- ☕ Message count: $3N + 3$
- ☕ Complexity: cao hơn
- ☕ Recovery: tự động hơn
- ☕ Ít dùng trong thực tế (replaced by SAGA)

SAGA Pattern – tổng quan

SAGA chia một distributed transaction thành chuỗi **local transaction** trên từng service, mỗi bước có một **compensating transaction** để hoàn tác nếu lỗi.

- ☕ Không dùng distributed lock → không blocking
- ☕ Mỗi service chỉ dùng local ACID transaction
- ☕ **Compensating transaction**: nghịch đảo của bước trước — hoàn tiền thay vì rollback
- ☕ Hai triển khai: **Orchestration** (điều phối tập trung) và **Choreography** (phân tán)

SAGA Orchestration

Orchestration SAGA dùng một **Orchestrator** trung tâm ra lệnh và giám sát từng bước — giống như nhạc trưởng điều phối dàn nhạc.

- Orchestrator gọi từng service theo thứ tự, nhận kết quả
- Nếu bước N fail → Orchestrator gọi compensating transaction của bước N-1, N-2...
- Logic workflow tập trung tại Orchestrator → dễ debug, dễ monitor
- Nhược điểm: Orchestrator là central dependency

Ví dụ: SAGA Orchestration — đặt vé máy bay

- ☕ Compensating transactions chạy *ngược chiều* khi có lỗi
- ☕ Mỗi bước lưu ID để compensating transaction biết phải hoàn tác gì

SAGA Choreography

Choreography SAGA không có Orchestrator — mỗi service tự lắng nghe event và quyết định hành động tiếp theo — như vũ công nhảy theo nhịp chung.

- ☕ Mỗi service publish event sau khi hoàn thành local transaction
- ☕ Service tiếp theo subscribe event và tự xử lý
- ☕ Compensating event được publish khi cần hoàn tác
- ☕ Không có central controller → không có single point of failure

Ví dụ: SAGA Choreography — luồng event

- ☕ Mỗi service chỉ quan tâm event liên quan đến mình
- ☕ Business rules phân tán vào từng service

So sánh Orchestration vs Choreography

Orchestration

- ☕ Control: tập trung tại Orchestrator
- ☕ Visibility: dễ trace luồng
- ☕ Coupling: service gọi Orchestrator
- ☕ Debug: dễ
- ☕ SPOF: Orchestrator
- ☕ Tốt khi: workflow phức tạp, nhiều bước

Choreography

- ☕ Control: phân tán qua events
- ☕ Visibility: khó trace (event chain)
- ☕ Coupling: loose coupling qua event bus
- ☕ Debug: khó hơn
- ☕ SPOF: event bus
- ☕ Tốt khi: service ít, workflow đơn giản

Distributed Consensus — Paxos

Paxos là giao thức đồng thuận cho phép cluster các node đạt được thỏa thuận về một giá trị dù có node bị lỗi — *nền tảng lý thuyết* của distributed systems.

- Gồm 3 vai: **Proposer** (đề xuất giá trị), **Acceptor** (chấp nhận hoặc từ chối), **Learner** (học giá trị đã đồng thuận)
- **Phase 1 (Prepare/Promise)**: Proposer gửi ballot number, Acceptor hứa không accept ballot cũ hơn
- **Phase 2 (Accept/Accepted)**: Proposer gửi giá trị, Acceptor accept nếu không có ballot mới hơn
- Đảm bảo safety dù minority node fail (chịu được $< N/2$ failures)

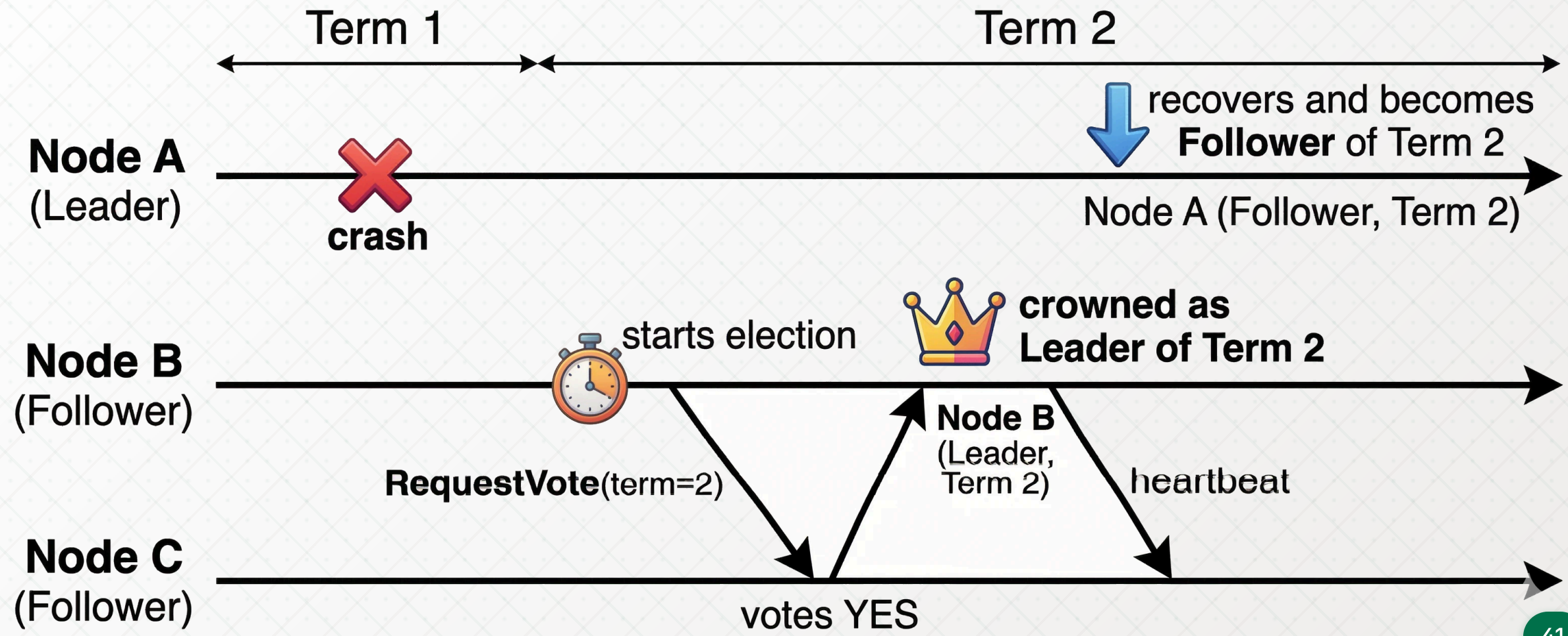
Distributed Consensus — Raft

Raft được thiết kế với mục tiêu *dễ hiểu* hơn Paxos trong khi đảm bảo cùng tính đúng đắn — phân chia rõ thành Leader Election, Log Replication, Safety.

- ☕ **Leader Election:** cluster bầu một Leader duy nhất qua voting (term-based)
- ☕ **Log Replication:** Leader nhận write request, replicate sang tất cả Follower trước khi commit
- ☕ **Safety:** một log entry chỉ commit khi được majority ($>N/2$) node xác nhận
- ☕ Chịu được $(N-1)/2$ node failure với cluster N node

Ứng dụng thực tế: etcd (Kubernetes), CockroachDB, TiKV (TiDB), Consul

Ví dụ: Raft Leader Election



So sánh Paxos vs Raft

Cả hai đều giải quyết bài toán distributed consensus nhưng Raft được thiết kế để *dễ implement đúng hơn*.

Paxos

- ☕ Thiết kế: lý thuyết tối ưu
- ☕ Understandability: *khó*
- ☕ Leader: không bắt buộc có leader cố định
- ☕ Log: không tường minh
- ☕ Implementation: error-prone

Raft

- ☕ Thiết kế: ưu tiên *dễ hiểu*
- ☕ Understandability: tốt hơn nhiều
- ☕ Leader: có Leader rõ ràng
- ☕ Log: replicated log tường minh
- ☕ Implementation: nhiều reference impl

Bài tập áp dụng

Bài tập 1: Thiết kế hệ thống tìm kiếm sản phẩm

Bạn được giao thiết kế tính năng tìm kiếm cho sàn thương mại điện tử có **50 triệu sản phẩm, 2 triệu DAU**.

Yêu cầu:

- 🍵 Tìm kiếm full-text theo tên, mô tả, thương hiệu
- 🍵 Hỗ trợ gõ sai chính tả (fuzzy) và gợi ý (typeahead)
- 🍵 Latency: < 200ms cho search, < 100ms cho typeahead
- 🍵 Ranking theo relevance và popularity

Câu hỏi thiết kế:

- 1 Bạn sẽ dùng công nghệ gì cho search layer? Tại sao không dùng SQL **LIKE** ?
- 2 Shard key cho Elasticsearch index là gì?
- 3 Làm thế nào để sync dữ liệu từ primary DB (PostgreSQL) sang Elasticsearch?
- 4 Thiết kế cache layer cho typeahead như thế nào?

Bài tập 2: Chọn chiến lược Replication và Sharding

Hệ thống **mạng xã hội** với đặc điểm: 100 triệu user, 80% traffic là đọc (timeline, profile), 20% là ghi (post, like, comment). Database hiện tại: PostgreSQL đơn node, đang quá tải.

Câu hỏi:

- 1 Nên dùng Replication hay Sharding trước? Tại sao?
- 2 Nếu Sharding — chọn shard key là `user_id` hay `timestamp`? Phân tích trade-off.
- 3 Vấn đề gì xảy ra khi user có 10 triệu follower (celebrity problem)?

Tình huống:

Metric	Hiện tại	Mục tiêu
Read QPS	50,000	500,000
Write QPS	12,000	120,000
Data size	2TB	20TB
Latency p99	800ms	100ms

Bài tập 3: ACID hay BASE? Phân tích tình huống

Với mỗi tình huống dưới đây, hãy quyết định nên dùng **ACID** hay **BASE**, và giải thích lý do.

Thảo luận: Với tình huống đặt hàng — nếu dùng BASE, vấn đề gì có thể xảy ra? Giải pháp?

Tình huống	Yêu cầu nghiệp vụ	ACID hay BASE?
Chuyển tiền giữa 2 tài khoản ngân hàng	Tuyệt đối không được mất tiền	???
Số lượng view của video YouTube	Hiển thị gần đúng, cập nhật mỗi 30s	???
Đặt hàng trên sàn thương mại (trừ kho, tạo đơn)	Không oversell, nhất quán inventory	???
Lưu lịch sử xem phim (Netflix history)	Cần lưu, chấp nhận delay vài giây	???
Bỏ phiếu bầu cử điện tử	Mỗi user chỉ được vote 1 lần	???

Bài tập 4: Thiết kế giao dịch đặt vé máy bay phân tán

Hệ thống đặt vé gồm 4 microservice độc lập. Khi user đặt vé, cần đảm bảo tính toàn vẹn dù bất kỳ service nào fail.

Luồng nghiệp vụ:

- 1 **InventoryService**: kiểm tra và giữ chỗ ngồi (seat reservation)
- 2 **PaymentService**: trừ tiền từ tài khoản user
- 3 **BookingService**: tạo booking record, generate e-ticket
- 4 **NotificationService**: gửi email xác nhận

Câu hỏi thiết kế:

- 1 Vẽ sơ đồ SAGA Orchestration cho luồng trên (bao gồm compensating transactions)
- 2 Vẽ sơ đồ SAGA Choreography tương đương (event names, subscribers)
- 3 Bạn chọn Orchestration hay Choreography? Lập luận dựa trên đặc điểm bài toán
- 4 Nếu PaymentService fail sau khi InventoryService đã giữ chỗ — compensating transaction cụ thể là gì?

Tóm tắt

- ☕ **Tìm kiếm dữ liệu lớn:** Inverted Index là nền tảng → Elasticsearch cho full-text search, Fuzzy Search và Typeahead cho UX tốt
- ☕ **Replication & Sharding:** Replication tăng availability và read throughput; Sharding tăng write throughput và vượt giới hạn storage
- ☕ **ACID vs BASE:** chọn theo yêu cầu nghiệp vụ — tài chính/y tế dùng ACID; scale lớn/chấp nhận stale dùng BASE
- ☕ **Giao dịch phân tán:** 2PC/3PC cho strong consistency; SAGA (Orchestration/Choreography) cho microservices; Paxos/Raft cho distributed consensus

Q & A
